GraphML-FS

Feature synthesis as the atomic search unit — controller / worker split, typed operator bank

Jiajun Zhu · UT Austin · May 2026

Recap

Prior deck: zhuconv.github.io/slides-hub/graphagent-4023

Where we left off — `GraphLoomer`

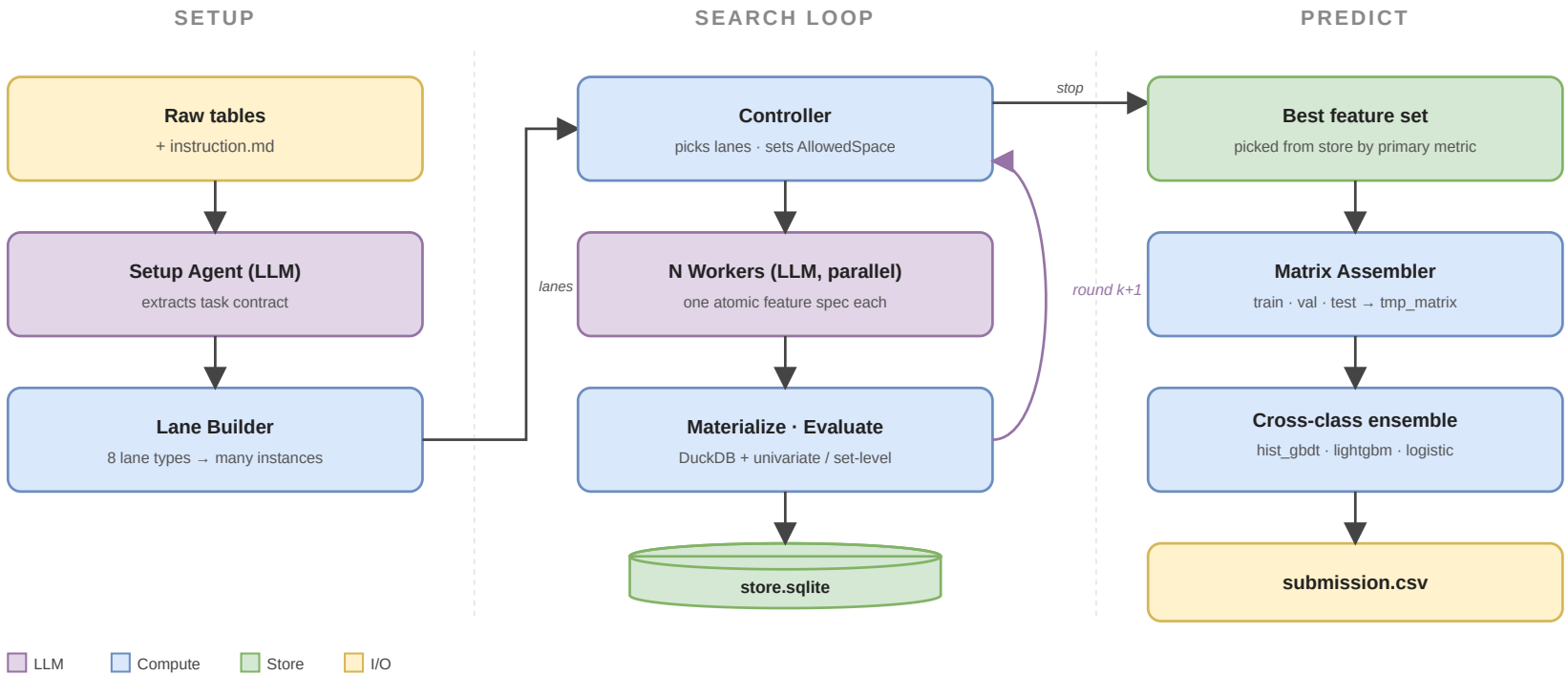
- Bilevel pipeline: outer model type, inner feature ops from a catalog
- **Whole** `solution.py` was the unit of search; one LLM agent looped Plan → Code → Execute → Refine
- 3rd of 4 end-to-end on GraphTestbed; failure-pattern features transferred to MLEvolve

What was missing

- **No parallelism inside a round** — one agent, one solution at a time
- **Errors hard to localise** — a feature bug crashed the whole pipeline
- **Free-form Python** outran the validator → debug churn dominated wall time

This deck: rebuild the harness around three architectural decisions, each fixing one of the gaps above.

At a glance — three stages, one loop



Setup runs once. Search loops until the controller stops. Predict materialises the best feature set into a single submission. Three pillars on the next three slides.

Pillar 1 — Controller ⊥ Worker, parallel rounds

What

- **Controller** — rule-based, no LLM. Enumerates lanes, sets `AllowedSpace`, decides stop.
- **Worker** — LLM. Stateless. Emits one `AtomicFeatureProposalBatch` for the lane it's given.
- **N workers per round, in parallel.**

Why it matters

- Round wall-time bounded by the slowest worker, not by total prompt count
- Errors localise to a single lane / spec
- Borrowed: AI-Build-AI's role separation; MLEvolve's per-node LLM expansion

GraphLoomer used one LLM looped on a whole pipeline — sequential, long context, blame hard to attribute. Splitting roles is the single biggest source of speed-up.

Pillar 2 — atomic feature spec is the search unit

What

A single search step produces:

```
spec = (lane, target_entity, path[],
        value_col, agg, window)
```

Canonicalised + hashed → deduped across rounds → persisted in `store.sqlite`.

Why it matters

- Materialize + univariate eval is **per spec** → embarrassingly parallel
- Round k+1 reuses rounds 0…k's matrices
- LLM mistakes scope to one feature, not the whole pipeline
- "What paid off" → next round's `AllowedSpace`

The downstream — impute → fit hist_gbdt / lightgbm / logistic → threshold sweep on val → CSV — is fixed code. The LLM never rewrites it.

Pillar 3 — typed operator bank: 8 lane *types*

Local lane types (single-table)

- **Pass-through** *(* `direct_numeric` *)*
- **Column transform** *(* `local_column_transform` *)* — log1p / clip / bucket
- **DSL expression** *(* `local_expr` *)* — expr over base cols
- **Hashed sparse cross** *(* `sparse_cross` *)* — cat × cat

Relational lane types (multi-table)

- **First-order group aggregate** *(* `shared_key_group_aggregate` *)* — 1-hop join + group-by
  e.g. for each `customer` *, count of their* `orders`
- **Co-neighbor snapshot** *(* `*_static_aggregate` *)* — 2-hop self-loop, no window
  e.g. for each `account` *, count of all txns it appeared in*
- **Co-neighbor windowed history** *(* `*_history` *)* — 2-hop self-loop, time-windowed
  e.g. for each `account` *, sum of txn amounts in last 7d*
- **Role-mixed co-neighbor history** *(* `*_mixed_history` *)* — 2-hop, two relation roles, windowed
  e.g. for each `account` *as sender, mean balance of recent recipients*

Why a closed, typed set

- Each type fixes **input dtype, output dtype, time policy, allowed aggs/windows**
- Validator rejects illegal specs **before** SQL runs
- LLM degrees of freedom **capped to options the validator understands**
- A new lane type is a **code change**, not an LLM choice

Why these names

The bank is dataset-agnostic; the `account_*` prefix is just a naming artifact from the first dataset's target entity (`Account` on ibm-aml). All three relational variants are **2-hop self-loops via a shared connector** — they differ only on (time-windowed?) and (same vs different relation on the two hops).

The Worker LLM picks SQL *within* a lane instance — never invents a new lane type. Sonnet still hits 37 % materialize-fail with this guardrail; free-form would be much worse.

Results — first on 3 / 4 GraphTestbed tasks

`graphfs-claude-sonnet-4-6` on the public leaderboard (lanczos/graphtestbed):

Task	Metric	**GraphML-FS**	rank	next-best
`figraph`	AUC-ROC	**0.895**	**#1**	0.890 (open-aibuildai)
`arxiv-citation`	AUC-ROC	**0.789**	**#1**	0.777 (open-aibuildai)
`ibm-aml`	F1 (minority)	**0.184**	**#1**	0.171 (open-aibuildai)
`ieee-fraud-detection`	AUC-ROC	0.921	#3	0.928 (aibuildai)

Same `claude-sonnet-4-6` as the AI-Build-AI runs — the harness is the differentiator, not the model.

Bigger picture — graph learning is operator learning

Where graph FMs hard-code the operator

- **GNN / OFA** — fixed local message-passing
- **Tokenizer Transformers** (Graphormer, OpenGraph) — PE / SVD / token bias
- **Hybrid GT** (GraphGPS) — local MPNN + global attention
- **PFN / G2T-FM** — graph adapter into a tabular FM

Pivot — operator scaling

First-class searchable modules:

- **Construction** — A, kNN, latent
- **Reduction** — sum / mean / attn
- **Multi-hop** — A^2, PPR, RW, heat
- **Structure** — LapPE, RWSE, motif
- **Head** — MLP / RF / XGB / PFN

NN's job: select, compose, calibrate, fuse.

GraphML-FS's 8 lane *types* are this thesis instantiated on the tabular-graph side. Generalising to end-to-end GraphFM training is the longer arc — operator scaling vs architecture scaling.

Next steps

1 — Three-axis bench

Per system × task × LLM: **performance**, **API cost**, **wall-time**. Compare GraphML-FS / MLEvolve / open-aibuildai across `sonnet-4-6 / opus-4-7 / gpt-5.4`. Hypothesis: typed bank caps tokens → lowest cost-per-point.

2 — Close the autoresearch ceiling

Task	now	ceiling	Δ
`arxiv-citation`	0.789	0.824	−0.035
`figraph`	0.895	0.940	−0.045
`ibm-aml`	0.184	0.591	−0.407

K-fold OOF · calibration · auto-NS · lane meta-prior.

3 — Ship as skills + tools

Drop-ins for **MLEvolve** / **open-aibuildai**: `graphfs.search` (matrix + lift) and `graphfs.lane_eval` (score one spec on val). Distribute as **MCP server** — zero host-controller change.

4 — Reminder · pre-NN vs post-NN aggregation

Current 8 lanes are **pre-NN** (column → downstream). Post-NN = wrap a model whose preds aggregate. Forces a differentiable / non-differentiable lane split. **Parked.**